Ciclo de Vida del Software

Versión 1.0 · 12 de noviembre de 2026

9.1 Análisis de requisitos y 9.2 Diseño

Enfoque propuesto: desarrollo incremental con entregas pequeñas, criterios de aceptación y revisión de evidencia. No se afirma que el equipo haya adoptado formalmente una metodología o un calendario.

Actores: cliente que solicita un turno, barbero que administra su agenda, responsable del local y equipo de desarrollo. El cliente no crea una cuenta en el recorrido actual; el barbero utiliza Supabase Auth.

Requisito	Situación conocida
Seleccionar servicio y horario	Interfaz implementada con catálogo y horarios fijos.
Calcular recargo VIP	Implementado en memoria y verificado en navegador.
Ingresar datos del cliente	Implementado con validaciones de formato insuficientes.
Autenticar al barbero	Código integrado; servicio real pendiente de validación por bloqueo previo de red.
Gestionar agenda	Datos de ejemplo; cancelación con etiqueta incorrecta.
Guardar y recuperar reservas	Pendiente de integración persistente.
Evitar dobles reservas y notificar	Pendiente de diseño, implementación y prueba.

Diseño existente: SPA Vue 3, Vue Router, stores reactivos, estilos Sass y cliente Supabase. Se propone evolucionar hacia operaciones persistentes y autorizadas en base de datos, manteniendo separados interfaz, autenticación y reglas de reserva.

Decisiones por definir: duración y solapamiento de turnos, fechas, capacidad por puesto o barbero, cancelaciones, reglas VIP, privacidad y notificaciones. Cada decisión debe registrar responsable, criterio y efecto sobre datos y pruebas.

Desarrollo, verificación y entrega

Desarrollo: trabajar sobre el checkout existente, revisar cambios previos y usar el lockfile para instalar. Mantener cambios acotados, revisables y vinculados a requisitos. No incluir valores sensibles en código, documentación o commits.

Verificación registrada: 19 comprobaciones de interfaz, 16 aprobadas y 3 fallidas, además de compilación exitosa. Las pruebas de la agenda utilizaron sesión simulada; no acreditan autenticación ni permisos de Supabase. Este documento reutiliza esos resultados y no presenta una nueva ejecución.

Defecto reproducido	Acción propuesta	Prueba de regresión
Teléfono con letras aceptado	Definir formato y validar antes de confirmar.	Rechazar abcdef sin bloquear formatos telefónicos admitidos.
Email abcd habilita registro	Validar formato y envío del formulario.	No enviar datos inválidos al servicio.
Cancelado aparece Pendiente	Representar todos los estados admitidos.	Cancelar y comprobar estado y etiqueta.

Puertas de calidad: requisitos aceptados, revisión de implementación, pruebas de la función y regresión afectada, compilación, configuración externa comprobada y documentación actualizada. Un resultado bloqueado debe conservarse como tal.

Entrega: generar un artefacto identificado por versión, revisar en un entorno de pruebas y publicar solo con autorización. Preparar recuperación del frontend y un plan independiente para cambios de base de datos. No hay una publicación productiva verificada en esta revisión.

Operación, mantenimiento y evolución

Operación: observar errores, disponibilidad y calidad del servicio con responsables definidos. Evitar datos personales innecesarios en registros y establecer retención y control de acceso.

Mantenimiento correctivo: resolver defectos reproducidos y añadir pruebas de regresión. Adaptativo: actualizar dependencias y compatibilidad. Evolutivo: integrar reservas persistentes, disponibilidad y notificaciones con sus criterios de aceptación.

Incremento propuesto	Resultado verificable
1. Correcciones de interfaz	Validaciones de teléfono y email, estados correctos y regresiones aprobadas.
2. Autenticación y permisos	Acceso real a Supabase, sesión y aislamiento entre usuarios verificados con cuenta de pruebas.
3. Reservas persistentes	Una reserva aparece en agenda y se conserva al recargar; control de concurrencia validado.
4. Preparación operativa	Notificaciones reales si se incluyen, respaldo restaurable, monitoreo y revisión de privacidad.

Trazabilidad: cada tarea debe vincular requisito, cambio, prueba y resultado. Registrar versión o commit, entorno, evidencia, defectos abiertos y decisión de aceptación. No se imponen fechas ni responsables personales desconocidos.

Cierre de una versión: comprobar criterios acordados y dejar explícitas las limitaciones. Retiro futuro: exportar o eliminar datos según las obligaciones aplicables, revocar accesos, desactivar integraciones y comunicar el cambio a los usuarios afectados.

Fuentes: README, package.json, vistas, router, stores, esquema SQL y documentación de resultados existente en Turnia. Propuestas y evidencia histórica se distinguen para evitar presentar el prototipo como un sistema operativo completo.